

PRACTICAL WEEK4

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ mkdir practical4
keerthi_priya@LAPTOP-OSGVCE7A:~$ cd practical4
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ nano task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ gcc task1.c -o task1
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ ./task1
Parent Process Started
Parent PID: 645
Child 1 started. PID = 646 Parent PID = 645
All 3 child processes have been created.
Child 2 started. PID = 647 Parent PID = 645

Using wait():
Child 3 started. PID = 648 Parent PID = 645
Child 1 completed. PID = 646
wait() detected child PID 646 finished with exit status 10

Using waitpid():
waitpid() detected child PID 646 finished with exit status 10
Child 2 completed. PID = 647
waitpid() detected child PID 647 finished with exit status 11
Child 3 completed. PID = 648
waitpid() detected child PID 648 finished with exit status 12
Parent process completed.
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ |
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");
    printf("\nUsing wait():\n");

    pid_t finished = wait(&status);

    printf("wait() detected child PID %d finished with exit status %d\n",
        finished, WEXITSTATUS(status));

    printf("\nUsing waitpid():\n");
}
```

```
keerthi_priya@LAPTOP-OSGV x + v task
GNU nano 8.7.1
{
    pid[i] = fork();

    if (pid[i] == 0)
    {
        printf("Child %d started. PID = %d Parent PID = %d\n",
            i + 1, getpid(), getppid());

        sleep(i + 1);

        printf("Child %d completed. PID = %d\n",
            i + 1, getpid());

        exit(10 + i);
    }
}

printf("All 3 child processes have been created.\n");
printf("\nUsing wait():\n");

pid_t finished = wait(&status);

printf("wait() detected child PID %d finished with exit status %d\n",
    finished, WEXITSTATUS(status));

printf("\nUsing waitpid():\n");

for (int i = 0; i < 3; i++)
{
    waitpid(pid[i], &status, 0);

    printf("waitpid() detected child PID %d finished with exit status %d\n",
        pid[i], WEXITSTATUS(status));
}

printf("Parent process completed.\n");

return 0;
}
```

TASK 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
keerthi_priya@LAPTOP-OSGV X + v
Child 3 started. PID = 648 Parent PID = 645
Child 1 completed. PID = 646
wait() detected child PID 646 finished with exit status 10

Using waitpid():
waitpid() detected child PID 646 finished with exit status 10
Child 2 completed. PID = 647
waitpid() detected child PID 647 finished with exit status 11
Child 3 completed. PID = 648
waitpid() detected child PID 648 finished with exit status 12
Parent process completed.
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ nano task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ nano task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ gcc task2.c -o task2
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ ./task2
Parent process: PID = 724
Child PID = 725
Parent sleeping...
Child process: PID = 725
Child process terminating...
Parent terminating...
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ nano task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ gcc task2.c -o task2
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ ./task2.c
-bash: ./task2.c: Permission denied
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ ./task2
Parent process: PID = 1042
Child PID = 1043
Parent sleeping...
Child process: PID = 1043
Child process terminating...
Parent terminating...
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ nano task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ gcc task2.c -o task2
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ ./task2
Child process terminating...
Zombie process removed.
keerthi_priya@LAPTOP-OSGVCE7A:~/practical4$ |
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());

        printf("Child process terminating...\n");

        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());

        printf("Child PID = %d\n", pid);

        printf("Parent sleeping...\n");

        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        wait(NULL);

        printf("Zombie process removed.\n");
    }

    return 0;
}
```